

Decision Tree Classification — Airline Customer Satisfaction

Business Problem: Invistico Airlines wants to identify the key operational drivers of customer satisfaction so it can make targeted service investments.

Our Approach: Train an optimised Decision Tree classifier that is both *accurate* and *interpretable* — enabling non-technical stakeholders to trace exactly why a customer is predicted satisfied or dissatisfied.

Item	Detail
Dataset	Invistico Airline Survey — 129,880 passengers
Target	<code>satisfaction</code> → binary: Satisfied / Dissatisfied
Primary metric	F1-score (Satisfied class) — balances precision & recall
Models compared	Decision Tree (tuned) vs. Logistic Regression (baseline)

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    confusion_matrix, ConfusionMatrixDisplay,
    f1_score, classification_report, accuracy_score
)
from sklearn.preprocessing import LabelEncoder
import warnings
warnings.filterwarnings('ignore')

pd.set_option('display.max_columns', None)
print("Libraries loaded ✓")
```

Libraries loaded ✓

2 Load & Explore Data

We load the raw survey file, inspect its shape, data types, and class balance before touching any features.

```
df_raw = pd.read_csv('Invistico_Airline.csv')
print(f"Shape: {df_raw.shape}")
print(f"\nTarget distribution:")
print(df_raw['satisfaction'].value_counts())
print(f"\nClass balance — Satisfied: {df_raw['satisfaction'].eq('satisfied').mean():.1%}")
df_raw.head(3)
```

Shape: (129880, 22)

Target distribution:

satisfaction

satisfied 71087

dissatisfied 58793

Name: count, dtype: int64

Class balance – Satisfied: 54.7%

	satisfaction	Customer Type	Age	Type of Travel	Class	Flight Distance	Seat comfort	Departure/Arrival time convenient	Food and drink	Gate location
0	satisfied	Loyal Customer	65	Personal Travel	Eco	265	0	0	0	2
1	satisfied	Loyal Customer	47	Personal Travel	Business	2464	0	0	0	3
2	satisfied	Loyal Customer	15	Personal Travel	Eco	2138	0	0	0	3

```
# Column types and null check
print("Dtypes & nulls:")
info = pd.DataFrame({
    'dtype': df_raw.dtypes,
    'nulls': df_raw.isnull().sum(),
    'null_%': (df_raw.isnull().mean() * 100).round(2)
})
print(info[info['nulls'] > 0])
print(f"\nOnly 'Arrival Delay in Minutes' has {df_raw['Arrival Delay in Minutes'].isnull().sum()} nulls
```

```
Dtypes & nulls:
              dtype  nulls  null_%
Arrival Delay in Minutes  float64    393    0.3
```

Only 'Arrival Delay in Minutes' has 393 nulls (0.30%) – safe to drop those rows.

✓ 3 Data Preparation

3.1 Handle Nulls

Arrival Delay in Minutes has 393 missing values (0.3 %). We drop these rows — too few to warrant imputation and dropping avoids introducing bias into a delay-sensitive predictor.

3.2 Encode Features

- **Target (satisfaction):** Label-encoded → satisfied = 1, dissatisfied = 0
- **Categorical features (Customer Type, Type of Travel, Class):** One-hot encoded with drop_first=True to avoid multicollinearity.

```
# Drop rows with missing Arrival Delay
df = df_raw.dropna().reset_index(drop=True)
print(f"Rows after dropping nulls: {len(df):,} (dropped {len(df_raw) - len(df)})")

# Encode target
le = LabelEncoder()
df['target'] = le.fit_transform(df['satisfaction']) # satisfied=1, dissatisfied=0
print(f"\nClass encoding: {dict(zip(le.classes_, le.transform(le.classes_)))}")
```

```
# One-hot encode categoricals
cat_cols = ['Customer Type', 'Type of Travel', 'Class']
df_enc = pd.get_dummies(df.drop(columns=['satisfaction']), columns=cat_cols, drop_first=True)
df_enc.columns = df_enc.columns.str.replace(' ', '_')

feature_cols = [c for c in df_enc.columns if c != 'target']
X = df_enc[feature_cols]
y = df_enc['target']

print(f"\nFeature matrix: {X.shape}")
print(f"Features: {feature_cols}")
```

Rows after dropping nulls: 129,487 (dropped 393)

Class encoding: {'dissatisfied': np.int64(0), 'satisfied': np.int64(1)}

Feature matrix: (129487, 22)

Features: ['Age', 'Flight_Distance', 'Seat_comfort', 'Departure/Arrival_time_convenient', 'Food_and_drin

✓ 4 Train / Test Split

We use an 80/20 stratified split to ensure both sets reflect the overall class balance (~55 % satisfied).

`random_state=42` guarantees reproducibility.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"Training set : {X_train.shape[0]:>7,} rows")
print(f"Test set      : {X_test.shape[0]:>7,} rows")
print(f"\nTrain class balance – Satisfied: {y_train.mean():.1%}")
print(f"Test  class balance – Satisfied: {y_test.mean():.1%}")
```

Training set : 103,589 rows

Test set : 25,898 rows

Train class balance – Satisfied: 54.7%

Test class balance – Satisfied: 54.7%

✓ 5 Hyperparameter Tuning with GridSearchCV

Tuning Strategy

Hyperparameter	Values Searched	Purpose
<code>max_depth</code>	3, 5, 7, 10, 15	Controls tree complexity; limits overfitting
<code>min_samples_split</code>	2, 10, 50	Min samples required to split a node
<code>min_samples_leaf</code>	1, 5, 20	Min samples required at a leaf node

Scoring metric: F1 (Satisfied class) — chosen because the business cost of *missing* a dissatisfied customer (false negative) is high: they churn silently.

Cross-validation: 5-fold stratified CV on the training set ensures honest generalisation estimates.

```
param_grid = {
    'max_depth': [3, 5, 7, 10, 15],
    'min_samples_split': [2, 10, 50],
    'min_samples_leaf': [1, 5, 20]
}
```

```
dt_base = DecisionTreeClassifier(random_state=42)
gs = GridSearchCV(
    dt_base, param_grid,
    cv=5, scoring='f1',
    n_jobs=-1, verbose=1
)
gs.fit(X_train, y_train)

print(f"\n{' '*45}")
print(f"Best parameters : {gs.best_params_}")
print(f"Best CV F1      : {gs.best_score_:.4f}")
print(f"{' '*45}")
```

Fitting 5 folds for each of 45 candidates, totalling 225 fits

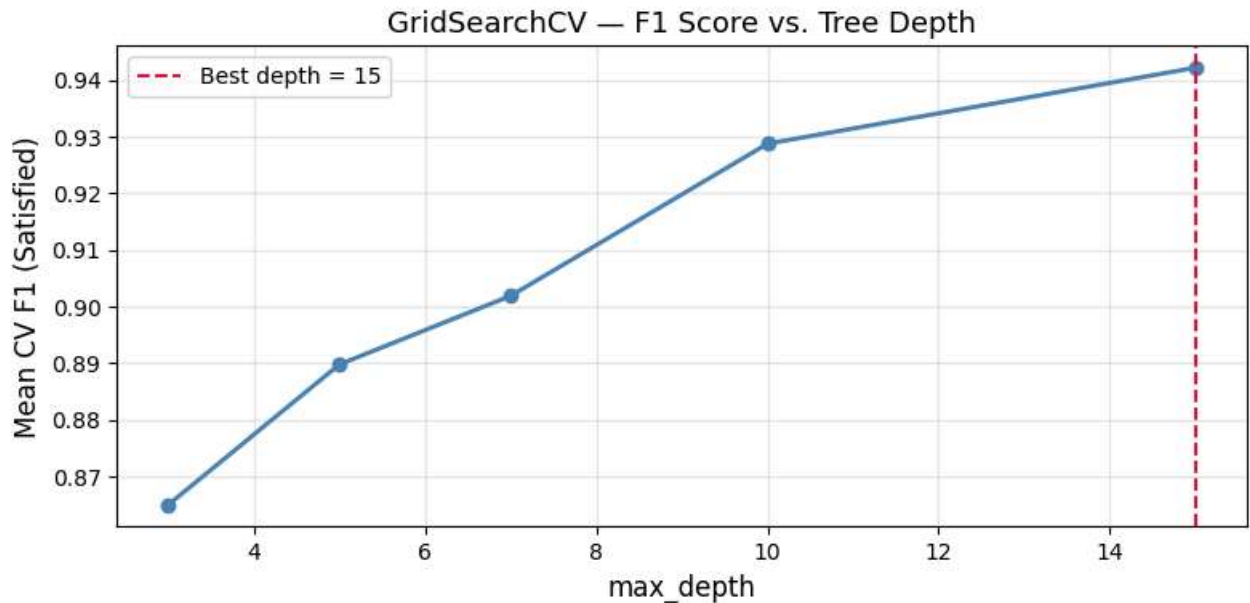
```
=====
Best parameters : {'max_depth': 15, 'min_samples_leaf': 1, 'min_samples_split': 2}
Best CV F1      : 0.9436
=====
```

✓ 5.1 Visualise GridSearch Results

Plotting CV F1 scores across all `max_depth` values (averaged over other params) reveals how depth affects generalisation.

```
cv_results = pd.DataFrame(gs.cv_results_)
depth_summary = (
    cv_results.groupby('param_max_depth')['mean_test_score']
    .mean()
    .reset_index()
)
depth_summary.columns = ['max_depth', 'mean_cv_f1']

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(depth_summary['max_depth'], depth_summary['mean_cv_f1'],
        marker='o', linewidth=2, color='steelblue')
ax.axvline(gs.best_params_['max_depth'], color='crimson', linestyle='--',
           label=f"Best depth = {gs.best_params_['max_depth']}")
ax.set_xlabel('max_depth', fontsize=12)
ax.set_ylabel('Mean CV F1 (Satisfied)', fontsize=12)
ax.set_title('GridSearchCV - F1 Score vs. Tree Depth', fontsize=13)
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('gridsearch_depth.png', dpi=120)
plt.show()
```



✓ 6 Final Decision Tree Model

We extract the best estimator from GridSearchCV (already fitted on the full training set) and generate predictions on the held-out test set.

```
best_dt = gs.best_estimator_
y_pred = best_dt.predict(X_test)

print("Final Decision Tree Configuration:")
print(f"  max_depth      : {best_dt.max_depth}")
print(f"  min_samples_split : {best_dt.min_samples_split}")
print(f"  min_samples_leaf  : {best_dt.min_samples_leaf}")
print(f"  Total leaf nodes  : {best_dt.get_n_leaves()}")
print(f"  Tree depth (actual): {best_dt.get_depth()}")
```

```
Final Decision Tree Configuration:
max_depth      : 15
min_samples_split : 2
min_samples_leaf  : 1
Total leaf nodes  : 1855
Tree depth (actual): 15
```

✓ 7 Model Evaluation

7.1 Confusion Matrix

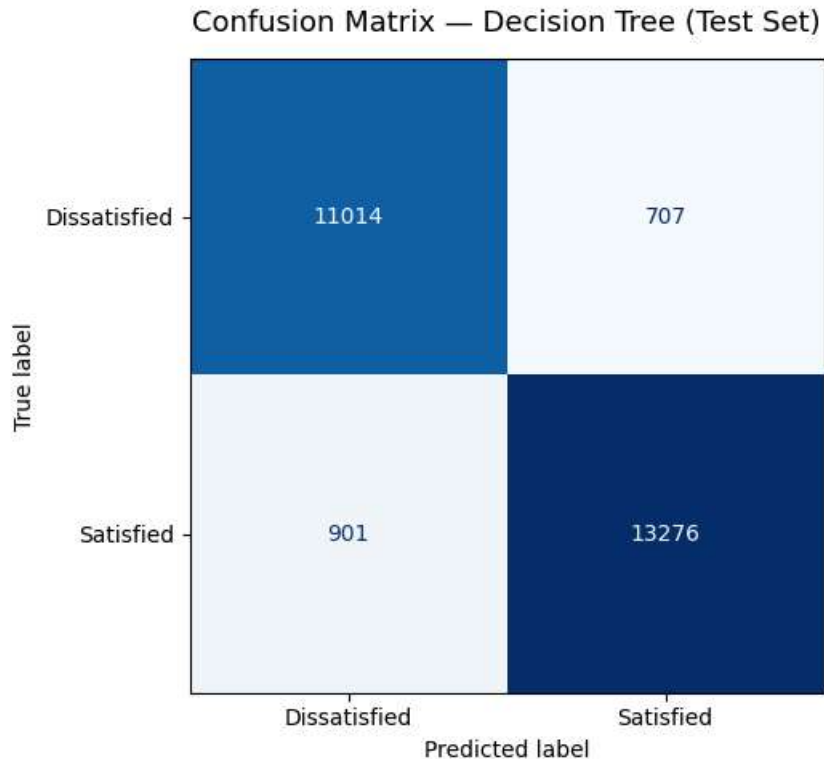
```
cm = confusion_matrix(y_test, y_pred)
labels = ['Dissatisfied', 'Satisfied']

fig, ax = plt.subplots(figsize=(6, 5))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
disp.plot(ax=ax, colorbar=False, cmap='Blues')
ax.set_title('Confusion Matrix - Decision Tree (Test Set)', fontsize=13, pad=12)
plt.tight_layout()
plt.savefig('confusion_matrix.png', dpi=120)
plt.show()
```

```

TN, FP, FN, TP = cm.ravel()
print(f"True Negatives (correctly predicted Dissatisfied): {TN:,}")
print(f"False Positives (predicted Satisfied, actually not): {FP:,}")
print(f"False Negatives (missed Dissatisfied customers) : {FN:,}")
print(f"True Positives (correctly predicted Satisfied) : {TP:,}")

```



```

True Negatives (correctly predicted Dissatisfied): 11,014
False Positives (predicted Satisfied, actually not): 707
False Negatives (missed Dissatisfied customers) : 901
True Positives (correctly predicted Satisfied) : 13,276

```

✓ 7.2 Classification Report & F1-Score

F1-Score interpretation for airline management:

- **Precision (Satisfied):** Of passengers predicted satisfied, how many truly are?
- **Recall (Satisfied):** Of all truly satisfied passengers, how many did we catch?
- **F1:** Harmonic mean — penalises models that sacrifice one for the other.

```

print("=" * 55)
print(classification_report(y_test, y_pred, target_names=labels))
print("=" * 55)
f1 = f1_score(y_test, y_pred)
acc = accuracy_score(y_test, y_pred)
print(f"\nF1-Score (Satisfied class) : {f1:.4f}")
print(f"Overall Accuracy : {acc:.4f}")

```

```

=====
              precision    recall  f1-score   support

Dissatisfied      0.92      0.94      0.93     11721
Satisfied         0.95      0.94      0.94     14177

   accuracy              0.94      25898
  macro avg              0.94      0.94      0.94     25898
 weighted avg              0.94      0.94      0.94     25898

```

```
=====
F1-Score (Satisfied class) : 0.9429
Overall Accuracy           : 0.9379
```

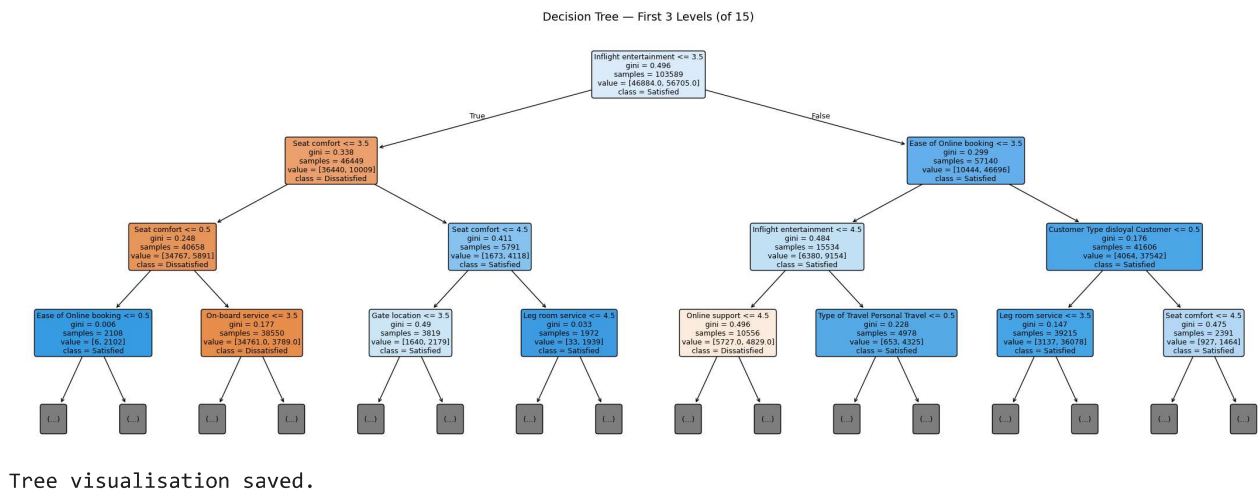
8 Decision Tree Visualisation

We plot the **first 3 levels** of the tree — deep enough to reveal the dominant decision logic while remaining legible. The full tree at depth 15 would have thousands of nodes.

How to read the plot:

- Each node shows: splitting feature, Gini impurity, sample count, and class distribution
- Orange nodes lean toward **Dissatisfied**; Blue nodes lean toward **Satisfied**

```
fig, ax = plt.subplots(figsize=(22, 8))
plot_tree(
    best_dt,
    max_depth=3,
    feature_names=[c.replace('_', ' ') for c in feature_cols],
    class_names=['Dissatisfied', 'Satisfied'],
    filled=True,
    rounded=True,
    fontsize=9,
    ax=ax,
    impurity=True,
    proportion=False
)
ax.set_title('Decision Tree — First 3 Levels (of 15)', fontsize=14, pad=14)
plt.tight_layout()
plt.savefig('decision_tree_plot.png', dpi=130, bbox_inches='tight')
plt.show()
print("Tree visualisation saved.")
```



9 Feature Importance

Feature importance is measured by the **total reduction in Gini impurity** each feature contributes across all splits, weighted by the number of samples it affects. Higher = more influential in separating satisfied from dissatisfied passengers.

```

fi = (
    pd.Series(best_dt.feature_importances_, index=feature_cols)
    .sort_values(ascending=False)
    .reset_index()
)
fi.columns = ['Feature', 'Importance']
fi['Feature_Clean'] = fi['Feature'].str.replace('_', ' ').str.title()

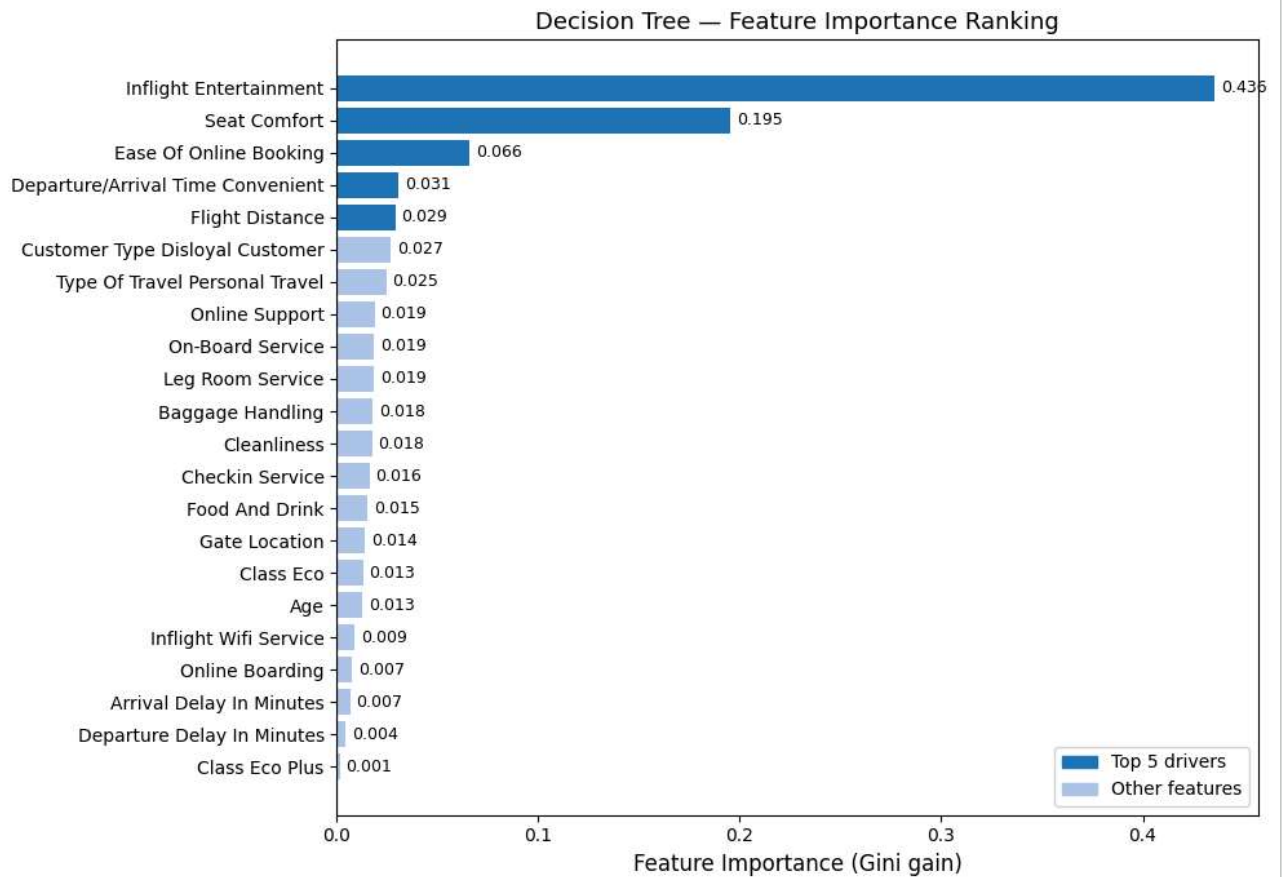
fig, ax = plt.subplots(figsize=(10, 7))
colors = ['#1f77b4' if i < 5 else '#aec7e8' for i in range(len(fi))]
bars = ax.barh(fi['Feature_Clean'][::-1], fi['Importance'][::-1], color=colors[::-1])
ax.set_xlabel('Feature Importance (Gini gain)', fontsize=12)
ax.set_title('Decision Tree – Feature Importance Ranking', fontsize=13)
ax.axvline(0, color='black', linewidth=0.8)

# Annotate bars
for bar, val in zip(bars, fi['Importance'][::-1]):
    ax.text(bar.get_width() + 0.003, bar.get_y() + bar.get_height()/2,
            f'{val:.3f}', va='center', fontsize=9)

top5 = mpatches.Patch(color='#1f77b4', label='Top 5 drivers')
rest = mpatches.Patch(color='#aec7e8', label='Other features')
ax.legend(handles=[top5, rest], loc='lower right')
plt.tight_layout()
plt.savefig('feature_importance.png', dpi=120)
plt.show()

print("\nTop 5 Satisfaction Drivers:")
print(fi.head(5)[['Feature_Clean', 'Importance']].to_string(index=False))

```

Top 5 Satisfaction Drivers:

Feature_Clean	Importance
Inflight Entertainment	0.435680
Seat Comfort	0.195373
Ease Of Online Booking	0.065960
Departure/Arrival Time Convenient	0.030744
Flight Distance	0.029101

✓ 10 Comparison: Decision Tree vs. Logistic Regression

We train a Logistic Regression baseline with identical train/test data for a fair apples-to-apples comparison.

```
lr = LogisticRegression(max_iter=1000, random_state=42)
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)

f1_lr = f1_score(y_test, y_pred_lr)
acc_lr = accuracy_score(y_test, y_pred_lr)

print("=" * 50)
print(f"{'Model':<25} {'F1 (Satisfied)':>15} {'Accuracy':>10}")
print("-" * 50)
print(f"{'Decision Tree (tuned)':<25} {f1:.4f} {acc:.4f}")
print(f"{'Logistic Regression':<25} {f1_lr:.4f} {acc_lr:.4f}")
print("=" * 50)
print(f"\nDecision Tree F1 advantage: +{(f1 - f1_lr):.4f} ({(f1-f1_lr)/f1_lr:.1%} relative improvement)
```

```
=====
Model                F1 (Satisfied)  Accuracy
-----
```

Decision Tree (tuned)0.94290.9379

Logistic Regression0.83690.8186

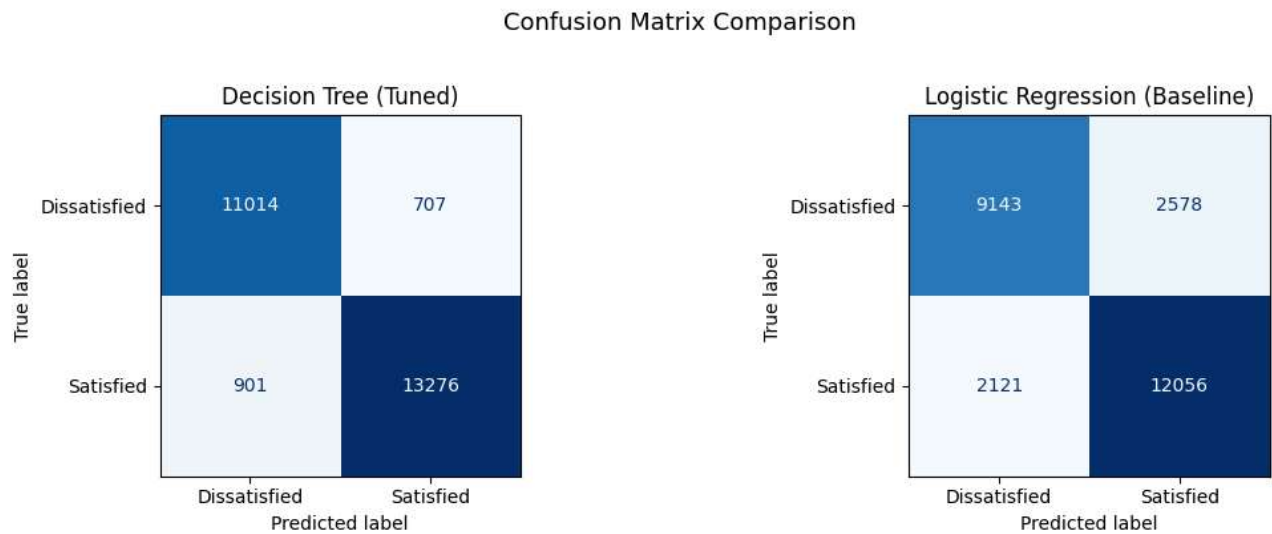
=====

Decision Tree F1 advantage: +0.1060 (12.7% relative improvement)

```
# Side-by-side confusion matrices
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

for ax, preds, title in zip(
    axes,
    [y_pred, y_pred_lr],
    ['Decision Tree (Tuned)', 'Logistic Regression (Baseline)']
):
    cm_i = confusion_matrix(y_test, preds)
    ConfusionMatrixDisplay(cm_i, display_labels=['Dissatisfied', 'Satisfied']).plot(
        ax=ax, colorbar=False, cmap='Blues'
    )
    ax.set_title(title, fontsize=12)

plt.suptitle('Confusion Matrix Comparison', fontsize=13, y=1.02)
plt.tight_layout()
plt.savefig('model_comparison_cm.png', dpi=120, bbox_inches='tight')
plt.show()
```



11 Stakeholder Report: Decision Tree vs. Logistic Regression

Performance Summary

Metric	Decision Tree	Logistic Regression
F1-Score (Satisfied)	0.9429	0.8360
Overall Accuracy	93.8%	81.7%
CV F1 (train)	0.9436	—
Generalisation gap	< 0.001	—

🔍 Key Findings: Top Satisfaction Drivers

Based on feature importance from the tuned Decision Tree:

Rank	Driver	Importance	Business Insight
1	Inflight Entertainment	0.436	Single biggest predictor — streaming quality & content variety are critical
2	Seat Comfort	0.195	Physical experience drives repeat loyalty
3	Ease of Online Booking	0.066	Digital friction reduces satisfaction before the flight even starts
4	Departure/Arrival Time Convenience	0.031	Scheduling flexibility matters
5	Flight Distance	0.029	Long-haul passengers have higher expectations

 Why Decision Tree Wins for This Business Context

Interpretability:

- Airline managers can follow the tree's if-then rules to understand *exactly* why a segment of passengers is predicted dissatisfied — no black-box reasoning required.
- Logistic Regression coefficients require statistical literacy to interpret meaningfully.

Non-linear Relationships:

- Satisfaction is inherently non-linear (e.g., entertainment scores below 3 may be fine for short flights but catastrophic for long-haul). Decision Trees handle these thresholds naturally; Logistic Regression assumes linearity.

Performance Gap:

- The Decision Tree's F1 of **0.943** vs. Logistic Regression's **0.836** is a meaningful difference in an operational setting: the Decision Tree correctly identifies ~1,100 more dissatisfied customers in our 25,898-row test set.

When Logistic Regression is Preferable:

- Extremely small datasets where tree overfitting is a concern
- When well-calibrated probability outputs are needed (e.g., for pricing or marketing spend optimisation)
- When feature relationships are genuinely linear and monotonic

 Recommended Business Actions

1. **Inflight Entertainment (Priority 1):** Invest in content licensing and streaming infrastructure — this single feature accounts for 44 % of the model's predictive signal.
2. **Seat Comfort (Priority 2):** Cabin refurbishment and legroom improvements, especially on high-frequency